

UNIVERSIDADE — TRABALHO 2

Controle de Concorrência em SQL

PostgreSQL 18 · Loja / Vendas

Aluno: Pedro Turik Firmino

Matrícula: 22200699

Disciplina: Infraestrutura para Gestão de Dados

Trabalho 2 — Controle de Concorrência em SQL

Disciplina: Infraestrutura para Gestão de Dados **SGBD:** PostgreSQL 18 (container Docker) **Contexto da aplicação:** Loja / vendas (`cliente` , `produto` , `pedido` , `item_pedido`)

Como reproduzir tudo: `bash scripts/run_all.sh` (sobe o banco, aplica esquema + dados, executa os cenários e a avaliação de desempenho). Veja o `README.md` para detalhes.

Visão geral do cenário

Quatro transações concorrentes que compartilham dados (linhas dos produtos 1 e 2 e a categoria `eletronicos`) e cumprem os papéis exigidos:

Transação	Papel exigido	O que faz	Bloqueios
T1	Causar Phantom Read	Relatório: conta <code>eletronicos</code> duas vezes; entre elas T4 insere	Implícito (leitura/snapshot), <code>READ COMMITTED</code>
T2	Causar 1 Deadlock	Transfere estoque: trava produto 1 → produto 2	Explícito <code>SELECT ... FOR UPDATE</code>
T3	Executar sem inconsistências	Venda: trava produto 2 → produto 1 (ordem inversa), registra pedido	Explícito <code>SELECT ... FOR UPDATE</code>
T4	Executar sem inconsistências	Cadastra novo produto <code>eletronicos</code> + reposição	Implícito (INSERT/UPDATE)

O **phantom** surge entre T1 e T4; o **deadlock** surge entre T2 e T3 (T2 é a vítima abortada, T3 sobrevive e confirma). Assim, um único escalonamento exercita os quatro comportamentos.

1. Script de criação das tabelas (`sql/01_schema.sql`)

```
DROP TABLE IF EXISTS item_pedido CASCADE;
DROP TABLE IF EXISTS pedido CASCADE;
DROP TABLE IF EXISTS produto CASCADE;
DROP TABLE IF EXISTS cliente CASCADE;

CREATE TABLE cliente (
    cliente_id SERIAL PRIMARY KEY,
    nome VARCHAR(120) NOT NULL,
    email VARCHAR(120) NOT NULL UNIQUE
);

CREATE TABLE produto (
    produto_id SERIAL PRIMARY KEY,
    nome VARCHAR(120) NOT NULL,
    categoria VARCHAR(60) NOT NULL,
    preco NUMERIC(10,2) NOT NULL CHECK (preco >= 0),
    estoque INTEGER NOT NULL CHECK (estoque >= 0)
);
CREATE INDEX idx_produto_categoria ON produto (categoria);

CREATE TABLE pedido (
    pedido_id SERIAL PRIMARY KEY,
    cliente_id INTEGER NOT NULL REFERENCES cliente (cliente_id),
    criado_em TIMESTAMP NOT NULL DEFAULT now(),
    status VARCHAR(20) NOT NULL DEFAULT 'ABERTO',
    total NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (total >= 0)
);

CREATE TABLE item_pedido (
    pedido_id INTEGER NOT NULL REFERENCES pedido (pedido_id),
    produto_id INTEGER NOT NULL REFERENCES produto (produto_id),
    quantidade INTEGER NOT NULL CHECK (quantidade > 0),
    preco_unitario NUMERIC(10,2) NOT NULL CHECK (preco_unitario >= 0),
    PRIMARY KEY (pedido_id, produto_id)
);
```

A restrição `CHECK (estoque >= 0)` garante a integridade que as transações concorrentes podem ameaçar (estoque negativo).

2. Script de inserção de dados (`sql/02_seed.sql`)

```
TRUNCATE item_pedido, pedido, produto, cliente RESTART IDENTITY CASCADE;

INSERT INTO cliente (nome, email) VALUES
  ('Ana Souza', 'ana@example.com'), ('Bruno Lima', 'bruno@example.com'),
  ('Carla Dias', 'carla@example.com'), ('Diego Alves', 'diego@example.com'),
  ('Elis Martins', 'elis@example.com');

-- ids 1..3 = 'eletronicos' (dados em comum das transacoes)
INSERT INTO produto (nome, categoria, preco, estoque) VALUES
  ('Notebook Pro', 'eletronicos', 4500.00, 30), -- produto_id = 1
  ('Smartphone X', 'eletronicos', 2999.90, 40), -- produto_id = 2
  ('Monitor 27"', 'eletronicos', 1200.00, 25), -- produto_id = 3
  ('Cadeira Gamer', 'moveis', 899.00, 15),      -- produto_id = 4
  ('Mesa Office', 'moveis', 650.00, 10);        -- produto_id = 5

INSERT INTO pedido (cliente_id, status, total) VALUES (1, 'FECHADO', 4500.00);
INSERT INTO item_pedido (pedido_id, produto_id, quantidade, preco_unitario)
VALUES (1, 1, 1, 4500.00);
```

Estado inicial relevante: **produto 1** → **estoque 30**, **produto 2** → **estoque 40**, **3 produtos** na categoria `eletronicos`.

3. Código das 4 transações

T1 — Relatório (causa Phantom Read) — `sql/transactions/t1_phantom.sql`

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED; -- nivel explicito
SELECT count(*) FROM produto WHERE categoria = 'eletronicos'; -- 1a leitura = 3
SELECT pg_sleep(:sleep1); -- janela para T4 inserir+commit
SELECT count(*) FROM produto WHERE categoria = 'eletronicos'; -- 2a leitura = 4 (PHANTOM)
SELECT sum(estoque) FROM produto WHERE categoria = 'eletronicos';
COMMIT;
```

T2 — Transferência de estoque (causa Deadlock) — `sql/transactions/t2_deadlock.sql`

```
BEGIN;
SELECT produto_id, estoque FROM produto WHERE produto_id = 1 FOR UPDATE; -- trava linha 1
UPDATE produto SET estoque = estoque - 5 WHERE produto_id = 1;
SELECT pg_sleep(:sleep1); -- deixa T3 travar linha 2
SELECT produto_id, estoque FROM produto WHERE produto_id = 2 FOR UPDATE; -- ESPERA -> DEADLOCK
UPDATE produto SET estoque = estoque + 5 WHERE produto_id = 2;
COMMIT; -- nao alcançado: PostgreSQL aborta (40P01) -> ROLLBACK
```

T3 — Venda (executa sem inconsistências) — `sql/transactions/t3_venda.sql`

```
BEGIN;
SELECT produto_id, estoque FROM produto WHERE produto_id = 2 FOR UPDATE; -- trava linha 2
UPDATE produto SET estoque = estoque - 1 WHERE produto_id = 2;
SELECT pg_sleep(:sleep1); -- forma o ciclo com T2
SELECT produto_id, estoque FROM produto WHERE produto_id = 1 FOR UPDATE; -- espera; apos T2 aborta
UPDATE produto SET estoque = estoque - 1 WHERE produto_id = 1;
INSERT INTO pedido (cliente_id, status, total) VALUES (2,'FECHADO',7499.90);
INSERT INTO item_pedido (pedido_id, produto_id, quantidade, preco_unitario)
VALUES (currval('pedido_pedido_id_seq'), 2, 1, 2999.90);
COMMIT; -- consistente
```

T4 — Cadastro/reposição (executa sem inconsistências) —

`sql/transactions/t4_cadastro.sql`

```
BEGIN;
SELECT pg_sleep(:sleep1); -- deixa T1 fazer a 1a leitura
INSERT INTO produto (nome, categoria, preco, estoque) -- novo item -> phantom de T1
VALUES ('Fone Bluetooth ...', 'eletronicos', 199.90, 50);
UPDATE produto SET estoque = estoque + 10 WHERE produto_id = 3;
SELECT count(*) FROM produto WHERE categoria = 'eletronicos';
COMMIT; -- consistente; torna o INSERT visível a T1
```

Cada transação tem ≥ 4 instruções e todas tocam **dados em comum** (produtos 1/2 e a categoria `eletronicos`), usando **bloqueios implícitos** (UPDATE/INSERT, snapshot de leitura) e **explícitos** (`SELECT ... FOR UPDATE`; `LOCK TABLE` é usado na avaliação de desempenho, item 5).

4. Tabela do cenário de escalonamento

Saída real de `scripts/scenario_full.sh` (T1 em READ COMMITTED). Estado inicial: produto 1 = 30, produto 2 = 40, 3 produtos `eletronicos`.

Ordem	T1 (relatório)	T2 (transferência)	T3 (venda)	T4 (cadastro)
1	BEGIN ; COUNT eletronicos			
	Resultado: 3			
	Observação: snapshot da 1ª leitura em READ COMMITTED			
2		BEGIN ; SELECT prod 1 FOR UPDATE ; UPDATE prod1 -5		
		Resultado: trava linha do produto 1		
		Observação: bloqueio exclusivo de linha adquirido (implícito p/ UPDATE, explícito via FOR UPDATE)		
3			BEGIN ; SELECT prod 2 FOR UPDATE ; UPDATE prod2 -1	
			Resultado: trava linha do produto 2	
			Observação: bloqueio exclusivo de linha adquirido	
4				BEGIN ; aguarda; INSERT eletronicos ; UPDATE prod3 +10 ; COMMIT
				Resultado: novo produto inserido e commitado
				Observação: sem conflito de lock; libera a versão nova para leitores
5		SELECT prod 2 FOR UPDATE		
		Resultado: ESPERA (bloqueado)		
		Observação: linha 2 está travada por T3 → T2 entra em espera		
6			SELECT prod 1 FOR UPDATE	
			Resultado: ESPERA (bloqueado)	
			Observação: linha 1 travada por T2 → fecha o	

Ordem	T1 (relatório)	T2 (transferência)	T3 (venda)	T4 (cadastro)
			ciclo (T2 → T3 → T2)	
7		DEADLOCK detectado (40P01) → ROLLBACK		
		Resultado: transação abortada (vítima)		
		Observação: PostgreSQL quebra o ciclo abortando T2; libera a linha 1		
8			adquire linha 1; UPDATE prod1 -1; INSERT pedido/item; COMMIT	
			Resultado: venda concluída e consistente	
			Observação: espera terminou ao T2 liberar o lock; sem inconsistência	
9	COUNT eletronicos; SUM; COMMIT			
	Resultado: 4 (PHANTOM)			
	Observação: 2ª leitura vê a linha inserida e commitada por T4 (passo 4)			

Estado final (consistente): produto 1 = 29, produto 2 = 39 (apenas a venda de T3 aplicada; a transferência de T2 foi revertida), produto 3 = 35 (reposição de T4), + novo produto `eletronicos` (estoque 50).

Comentário sobre bloqueios e esperas

- **Bloqueios implícitos:** todo `UPDATE / INSERT` adquire bloqueio exclusivo na linha afetada até o fim da transação; leituras em `READ COMMITTED` usam *snapshot por comando*.
- **Bloqueios explícitos:** `SELECT ... FOR UPDATE` trava antecipadamente as linhas dos produtos 1 e 2, o que é exatamente o que cria a espera mútua (passos 5–6).
- **Espera/Deadlock:** T2 espera a linha 2 (de T3) e T3 espera a linha 1 (de T2). O detector de deadlock do PostgreSQL (`deadlock_timeout`, padrão 1s) identifica o ciclo e aborta uma vítima (T2), garantindo progresso. T3 prossegue assim que o lock é liberado.
- **Phantom:** T1, em `READ COMMITTED`, enxerga em sua 2ª leitura uma linha que **não existia** na 1ª (inserida e commitada por T4). Em `SERIALIZABLE` isso não ocorre (ver §5).

5. Avaliação de desempenho

Foram escolhidas **duas abordagens** (a spec pede ≥ 2):

1. **Tempo de execução** de um lote concorrente.
2. **Número de tentativas de execução** (retries por *serialization failure* 40001 e por *deadlock* 40P01).

Carga (`scripts/perf_isolation.sh`): **4 workers concorrentes × 10 transações** = 40 transações por estratégia. Cada transação lê por predicado (`COUNT` da categoria `eletronicos`) e em seguida escreve nos produtos 1 e 2 (rw-conflito clássico de SSI), com `pg_sleep(0.02)` para alargar a janela de conflito.

5.1 Comparação de níveis de isolamento e mecanismos de bloqueio

Resultado de uma execução típica (`logs/run_perf.log`):

Estratégia	Txs	Tentativas	Retries	Tempo (ms)
READ COMMITTED (bloqueio implícito de linha)	40	40	0	~1010
SERIALIZABLE (SSI)	40	~142–150	~102–110	~4200–4500
READ COMMITTED + LOCK TABLE ... IN EXCLUSIVE MODE	40	40	0	~1030

Os números variam um pouco a cada execução; rode `scripts/perf_isolation.sh` para reproduzir.

5.2 Análise

- **Concorrência × consistência (READ COMMITTED)**: alta vazão e **zero retries**, porque conflitos são resolvidos por bloqueio de linha (um espera o outro). Em compensação, READ COMMITTED **permite phantom read** (demonstrado por T1) e anomalias de escrita não evitadas por locks explícitos.
- **SERIALIZABLE**: oferece a **maior garantia de consistência** (serializável de verdade, via *Serializable Snapshot Isolation*) e **elimina o phantom** — em §4 a variante SERIALIZABLE de T1 lê 3 nas duas vezes. O custo é alto: **~102–110 retries** e tempo **~4× maior**, pois transações conflitantes abortam com 40001 e precisam ser reexecutadas pela aplicação.
- **LOCK TABLE IN EXCLUSIVE MODE**: serializa o acesso por **bloqueio de tabela**. Não há retries (como em READ COMMITTED) e a consistência é forte para a operação, mas ao custo de **serialização total** da tabela (menor concorrência sob carga maior). É o oposto do otimismo do SERIALIZABLE: pessimista e determinístico.
- **SELECT ... FOR UPDATE** (usado em T2/T3): bloqueio pessimista **por linha**, granularidade melhor que `LOCK TABLE`. Garante leitura-para-atualização sem perder atualizações; pode, porém, gerar **deadlock** quando linhas são travadas em ordens opostas (exatamente o caso de T2×T3).

Conclusão: não há estratégia universalmente melhor. READ COMMITTED + locks explícitos de linha (`FOR UPDATE`) dá o melhor equilíbrio para operações pontuais (venda, transferência). SERIALIZABLE é indicado quando a aplicação exige ausência de anomalias (relatórios/consistência forte) e está preparada para **reexecutar** transações abortadas. `LOCK TABLE` é o recurso mais grosseiro, útil em lotes administrativos onde a serialização total é aceitável.

Anexos / reprodução

- `docker-compose.yml` — serviço PostgreSQL 18.
- `scripts/run_all.sh` — pipeline completo (init + 3 cenários + desempenho).
- `scripts/scenario_phantom.sh`, `scenario_deadlock.sh`, `scenario_full.sh`.
- `scripts/perf_isolation.sh` — avaliação de desempenho.
- Logs de cada execução em `logs/`.